

Taller Data Warehouse y Dashboard

En este taller se construye un pipeline *end-to-end*, que inicia en la extracción desde una base de datos relacional, el proceso de transformación y almacenamiento en un data warehouse, y la consulta por parte de una herramienta de exploración y visualización.

Entregable

- Fecha de entrega: Viernes 25 de Septiembre
- Cantidad de estudiantes por grupo: 1. Individual
- Contenido del entregable: Todo el proyecto completo en archivo comprimido [Taller-DW.zip](#)

Contexto

Generalmente las organizaciones almacenan sus datos de transacciones en bases de datos relacionales RDBMS. Estos sistemas orientados a files almacenan en tablas las transacciones rutinarias. Por ejemplo, las interacciones de compras, clientes, proveedores, empleados, productos, categorías y demás. Los analistas de negocio requieren obtener información general a partir de estos datos transaccionales. Sin embargo, los sistemas transaccionales presentan dificultades al consultar y procesar grandes cantidades de filas, puesto que fueron diseñados para optimizar las transacciones y búsquedas de pocos registros en sus tablas.

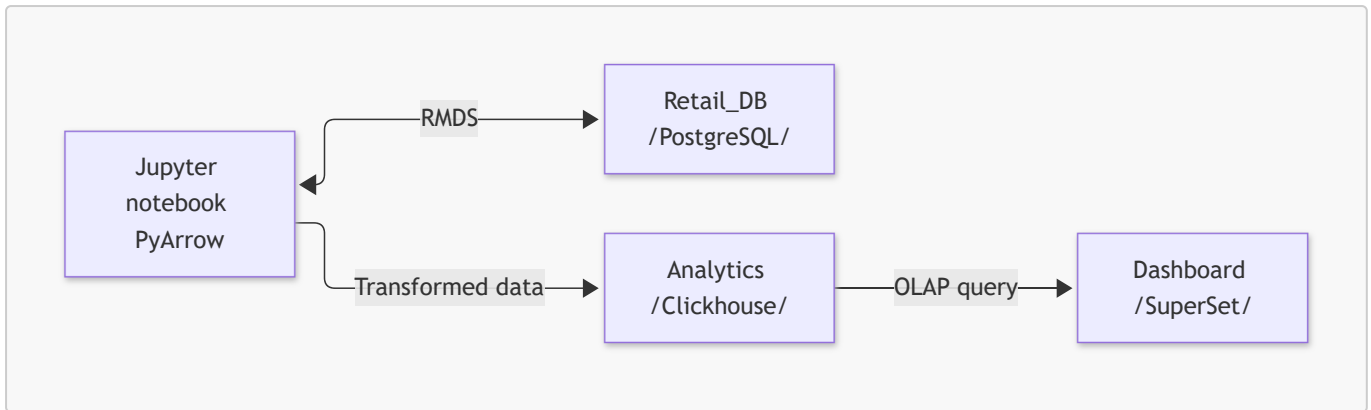
Los sistemas de analítica permiten acceder a información refinada (ultra-procesada) como resultado de secuencias de procesos, transformaciones y análisis. Las transformaciones se componen de múltiples etapas y tareas, que siguen el patrón ETL o ELT según el caso de uso y la arquitectura. Generalmente, los resultados o *insights* se presentan a través de tableros o *dashboards*, los cuales son fáciles de entender y manipular por los usuarios de negocios.

Este taller presenta un caso funcional del proceso de inicio a fin. Desde los datos almacenados en un sistema transaccional, su procesamiento y manipulación, y la consulta y análisis en un tablero útil al usuario final.

Componentes del taller

Al finalizar, este taller se compondrá de cuatro contenedores:

- Visualización: El *dashboard* en Superset [Apache Superset](#) para visualizar las consultas de negocio.
- OLAP: El sistema de almacenamiento de analítica en ClickHouse [Fuente a Clickhouse](#) que contendrá la base de datos de analítica.
- OLTP: El sistema de almacenamiento transaccional estará en un motor PostgreSQL.
- Jupyter notebook: tiene la responsabilidad de orquestar el flujo y transformación de datos entre la base de datos transaccional y la de analítica



Resultados esperados

Al terminar el taller tendremos una estructura de archivos así:



Establecer el Ambiente de Trabajo

Inicie el motor de Podman

```
podman machine start
```

Actividades del taller

1 Crear la Base de Datos Transaccional

1.1 Crear el Esquema de la Base de Datos Transaccional

📁 Utilice el archivo `02_sample_data.sql`, adjunto a este documento, que contiene los datos sintéticos iniciales. Construya otro archivo llamado `01_schema.sql` que contendrá el DDL de la base de datos en PostgreSQL. Guarde este archivo de esquema en el directorio `postgresql/init`.

1.2 Crear el Contenedor de PostgreSQL

Cree un archivo que contiene la imagen personalizada de PostgreSQL como `postgresql/Dockerfile` con el siguiente contenido. La base de datos se llamará **retail_db** y asigna variables con el nombre del usuario y su contraseña.

```
FROM postgres:14

# Asignar variables de entorno
ENV POSTGRES_DB=retail_db
ENV POSTGRES_USER=lab_user
ENV POSTGRES_PASSWORD=lab_pass

# Copia los scripts de esquema y datos
COPY ./init/ /docker-entrypoint-initdb.d/
```

⚠ Importante: Con fines didácticos, asignamos las credenciales como nombres de usuario y contraseña directamente en el archivo de construcción. En entornos de producción, las credenciales se administran en un Sistema de Gestión de Identidades como *Microsoft Entra ID* o *Identity and Access Management* de AWS. Para mayor información consulte [Enlace Identity Access Management](#).

Cree un archivo para el compose engine, llamado `compose.yaml` donde se ensamblarán los recursos del taller. En este **compose** agregue un servicio de postgresql para la construcción del contenedor.

```
services:

  postgres_server:
    build: ./postgresql
    container_name: postgres_server
    restart: unless-stopped
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```

healthcheck:
  test: ["CMD-SHELL", "pg_isready -U lab_user -d retail_db"]
  interval: 10s
  timeout: 5s
  retries: 5

volumes:
  postgres_data:

```

1.3 Crear un Contenedor de Jupyter Notebook

Para dirigir el flujo de datos usaremos un Notebook de Jupyter para acceder a los datos de *retail_db*.

```

services:
  ...
  jupyter:
    image: jupyter/scipy-notebook:python-3.10
    container_name: taller_jupyter
    ports:
      - "8888:8888"
    volumes:
      - ./notebooks:/home/jovyan/work
    environment:
      JUPYTER_TOKEN: taller_dw
    depends_on:
      - clickhouse_server
      - postgres_server
    command: >
      bash -c "
        pip install -r /home/jovyan/work/requirements.txt && start-notebook.sh
      "

```

Cree un archivo nombrado `notebooks/taller_clickhouse.ipynb` que contendrá las operaciones y transformaciones sobre los datos.

 Cree un archivo llamado `notebooks/requirements.txt` con las bibliotecas necesarias para conectarse a PostgreSQL `sqlalchemy`, y otras herramientas básicas para la manipulación de datos como `matplotlib` y `pandas`. Luego agregaremos más bibliotecas para la conexión a ClickHouse y otras herramientas específicas para la transformación de datos.

En este punto ya es posible lanzar los dos contenedores, PostgreSQL y Jupyter, y establecer una conexión desde el notebook de jupyter a la base de datos *retail_db*. Para esto usamos una instancia de **engine** como lo muestra el siguiente código en Python:

```

from sqlalchemy import create_engine

postgres_engine = create_engine(
    "postgresql://lab_user:lab_pass@postgres_server:5432/retail_db",

```

```
        echo=False,  
        future=True  
    )  
  
    postgres_engine
```

1.4 Consultar la Base de Datos *retail_db*

Luego podemos crear una consulta que nos muestre las tablas de *retail_db*.

```
from sqlalchemy import text  
  
query = """  
    SELECT table_name  
    FROM information_schema.tables  
    WHERE table_schema = 'public'  
    ORDER BY table_name;  
    """  
  
with postgres_engine.connect() as conn:  
    result = conn.execute(text(query))  
  
    for row in result:  
        print(row[0])
```

El resultado esperado es la lista de tablas, según el archivo de datos será:

categories

customer_customer_demo

customer_demographics

customers

employee_territories

employees

order_details

orders

products

region

shippers

suppliers

territories

1.5 Consulta de clientes de retail_db

 Consultemos algunos clientes. Construya una consulta en SQL que muestre el id, la empresa y el país de los primeros 10 clientes de la tabla *customers*

customer_id	company_name	country
C0001	Anderson Foods	USA
C0002	Lopez Grocery	Colombia
...		

1.6 Consulte cada una de las tablas

 Cree una consulta para cada tabla. Limite el resultado a solo 10 registros para evitar cuellos de botella. Escriba y ejecute cada consulta en una celda del notebook.

1.7 Consulta compuesta JOIN de órdenes, clientes y empleados

 Construya una consulta que relacione *orden cliente* y *empleado* para identificar empleados que han vendido a un cliente en particular, y el costo del flete para una orden. El resultado de la consulta será como el siguiente:

order_id	company_name	employee	order_date	freight
1	Andersson Foods	Margaret	2026-01-09	7.11
2	Dupont Market	Olivia	2026-01-08	7.61
3	Dupont Market	James	2026-01-29	61.27
...				

2. Crear un Data Warehouse en ClickHouse

ClickHouse es un motor de base de datos de analítica, ampliamente usado en la industria, que permite construir dashboards y análisis en tiempo real sobre grandes volúmenes de datos. [Fuente ClickHouse](#)

Tras bambalinas, ClickHouse utiliza un sistema columnar en el cual la información se organiza en columnas en vez de filas. En los sistemas tradicionales orientados a filas (row-based), cada registro se almacena completo forma contigua, lo que facilita las operaciones transaccionales que requieren acceder o modificar la fila entera. En cambio, los sistemas columnar almacenan cada columna de manera independiente, y agrupa los registros de un mismo atributo en bloques contiguos de memoria. El acceso a memoria es más eficiente y reduce los fallos de caché a lo cual se le llama *localidad de datos en caché*.

Esta organización impacta directamente en el rendimiento de las consultas analíticas. Generalmente las consultas acceden a pocas columnas para calcular agregaciones, como SUM o AVG, y así el motor accede únicamente a las columnas en el disco, y evita cargar datos innecesarios. Así reduce significativamente la carga I/O, considerado el principal desafío en los sistemas de procesamiento de gran escala.

Otro aspecto importante es la compresión. Dado que los datos de una misma columna son relativamente homogéneos, por ejemplo son fechas o categorías, los algoritmos de compresión son más eficientes comparado con la compresión de filas. Técnicas como *run-length encoding* RLE o *delta encoding* reducen el tamaño en disco e incrementan el flujo de lectura porque procesan menor cantidad de bytes.

Adicionalmente, los sistemas columnares están diseñados para aprovechar la ejecución vectorizada, en vez de procesar una fila a la vez, mediante el uso de instrucciones **SIMD** (Single Instruction, Multiple Data) de la CPU. Estas operaciones incrementan el rendimiento de agregaciones y filtros. Es también relevante el concepto de *late materialization* donde primero filtra las columnas relevantes y posteriormente reconstruye las *filas* necesarias.

El almacenamiento columnar se optimiza para cargas de trabajo de analítica, donde predominan las lecturas intensivas, las agregaciones y escaneo de grandes volúmenes de datos. No es adecuado para escenarios transaccionales que requieren inserciones y actualizaciones frecuentes al nivel de fila.

Ahora crearemos un contenedor para ClickHouse. Al archivo de compose, agregue la definición del nuevo servicio:

```
services:
  ...

  clickhouse_server:
    image: clickhouse/clickhouse-server:24.8
    container_name: clickhouse_server
    restart: always
    ports:
      - "8123:8123"
      - "9000:9000"
    volumes:
      - clickhouse_data:/var/lib/clickhouse
      - ./clickhouse/config:/etc/clickhouse-server/users.d
      - ./clickhouse/init:/docker-entrypoint-initdb.d
    environment:
      CLICKHOUSE_DB: analytics
      CLICKHOUSE_USER: admin
      CLICKHOUSE_PASSWORD: admin123
      CLICKHOUSE_DEFAULT_ACCESS_MANAGEMENT: 1

  ...
volumes:
  clickhouse_data:
```

Contenido del archivo `clickhouse_config/admin-user.html`:

```
<clickhouse>
  <users>
    <admin>
      <profile>default</profile>
    <networks>
```

```

    <ip>::/0</ip>
  </networks>
  <password>admin123</password>
  <quota>default</quota>
  <access_management>1</access_management>
</admin>
</users>
</clickhouse>

```

Contenido del archivo `clickhouse_config/default-user.html`:

```

<clickhouse>
  <!-- Docs: <https://clickhouse.com/docs/en/operations/settings/settings_users/>
  -->
  <users>
    <!-- Remove default user -->
    <default remove="remove">
    </default>

    <admin>
      <profile>default</profile>
      <networks>
        <ip>::/0</ip>
      </networks>
      <password><![CDATA[admin123]]></password>
      <quota>default</quota>
      <access_management>1</access_management>
    </admin>
  </users>
</clickhouse>

```

Contenido del archivo `

 Agregue `clickhouse-connect` y `pyarrow` al archivo de requerimientos de Jupyter Notebook. Luego, reconstruya las imágenes y los contenedores necesarios para reiniciar los cambios.

2.1 Creación de tablas en ClickHouse

Para esta sección se puede utilizar una herramienta de bases de datos como [DBeaver](#) y crear conexiones a ClickHouse. Para esto, instale el driver siguiendo las instrucciones de [Connect DBeaver to ClickHouse](#)

Creemos una base de datos con nombre *analytics* y una tabla donde registraremos los eventos e interacciones de navegación de los usuarios por un catálogo de productos, por ejemplo Amazon.

```

CREATE DATABASE IF NOT EXISTS analytics;

USE analytics;

CREATE TABLE eventos

```

```
(
    evento_ts DateTime,
    user_id UInt64,
    session_id String,
    evento_tipo LowCardinality(String),
    producto_id UInt32,
    categoria String,
    precio Float32
)
ENGINE = MergeTree
ORDER BY (evento_ts, user_id)
PARTITION BY toDate(evento_ts);
```

Luego insertamos algunos datos de prueba en la tabla *events*:

```
INSERT INTO analytics.events VALUES
('2025-01-01 10:00:00',1,'s1','vista',1001,'electronica',499.99),
('2025-01-01 10:01:00',1,'s1','carrito',1001,'electronica',499.99),
('2025-01-01 10:02:00',1,'s1','compra',1001,'electronica',499.99),
('2025-01-01 10:05:00',2,'s2','vista',2001,'libros',19.99);
```

ClickHouse implementa almacenamiento columnar en forma física mediante la separación de datos en archivos independientes dentro de unidades llamadas *parts*. Cada part contiene los datos de un subconjunto de filas, y cada columna se almacena en archivos binarios *.bin* junto con los archivos de marcas *mkr* o comprimidos *.cmrk3* para indexar las posiciones dentro de los datos. Al leer una columna, el motor de ClickHouse accede directamente al archivo correspondiente de la columna sin necesidad de leer las demás.

Clickhouse no utiliza índices tradicionales como B-tree. Utiliza un índice primario basado en el ordenamiento físico de los datos. Observe la instrucción **ORDER BY** en el script de creación de la tabla. El índice consiste en marcas *marks* que apuntan a posiciones en los archivos de columnas de cada cierta cantidad de filas (por defecto 8192). Cuando se ejecuta la búsqueda, el motor utiliza las marcas para saltar bloques completos que no cumplen las condiciones **WHERE**, esta operación se denomina *data skipping*. Data skipping es un mecanismo que utilizan regularmente los sistemas columnares y completamente distinto a los de acceso puntual de los sistemas OLTP.

ClickHouse también particiona los datos y los organiza de forma física en directorios según se defina la función de partición **PARTITION BY**. La partición permite evitar grandes volúmenes de datos de forma eficiente y reducir el espacio de búsqueda durante una consulta. El mecanismo de partición es complementario al índice de orden y permite el *partition pruning*, una optimización importante para los sistemas analíticos distribuidos. En el ejemplo se particionan los datos por el campo *event_time*. Verifique la estructura de los archivos de datos en */var/lib/clickhouse/data/* de su contenedor.

2.2 Conexión a la base de datos analytics de ClickHouse

Ahora crearemos una conexión a ClickHouse desde Python. En el notebook de Jupyter, en una nueva celda y complete los datos de la conexión

```
# conectarse a la base de datos de clickhouse
import clickhouse_connect
click_client = clickhouse_connect.get_client(
    host=<host_ip or docker id>,
    port=<port number>,
    username=<username>,
    password=<pass>,
    database=<db_name>
)
```

Luego consulte los datos de la tabla *eventos*

```
df = click_client.query_df("SELECT * FROM eventos LIMIT 10")
df
```

Consulte la documentación adicional para construir consultas más elaboradas en [Querying data with ClickHouse](#)

2.3. Medir el rendimiento de ClickHouse

En esta sección experimentaremos las capacidades de ClickHouse como motor de analítica mediante la creación y consulta de datos sintéticos.

Primero, crearemos una tabla nueva:

```
# crea una tabla *benchmark_eventos* y la llena con millones de datos

create_sql = """
CREATE TABLE IF NOT EXISTS benchmark_eventos
(
    evento_ts DateTime,
    usuario_id UInt32,
    producto_id UInt32,
    precio Float32
)
ENGINE = MergeTree
PARTITION BY toDate(evento_ts)
ORDER BY (usuario_id, evento_ts)
"""

res = click_client.command(create_sql)

print(res.summary)
```

Lea el detalle de los **ENGINE** de ClickHouse en la documentación [MergeTree table engine](#) y del [método command](#)

```
df = click_client.query_df("DESCRIBE TABLE benchmark_eventos")
df
```

Luego, insertamos los datos para el análisis comparativo:

```
import time
start = time.time()

insert_sql = """
INSERT INTO benchmark_eventos
SELECT
    now() - number % 10000,
    rand() % 100000,
    rand() % 1000,
    rand() % 500
FROM numbers(100000000)
"""

res = click_client.command(insert_sql)

end = time.time()

print("Insertadas 100M filas en:", round(end - start, 2), "segundos!")
print(res.summary)
```

Ahora, mediremos el tiempo de ejecución de una consulta con agregaciones sobre la tabla:

```
# consultas a la tabla
select_sql = """ SELECT
    producto_id,
    count() AS ventas,
    avg(precio) AS precio_promedio
FROM benchmark_eventos
GROUP BY producto_id
ORDER BY ventas DESC
LIMIT 10
"""

start = time.time()

df = click_client.query_df(select_sql)

end = time.time()

print("Duración de la consulta:", round(end - start, 4), "seconds")

df
```

Después, mediremos el tiempo de ejecución al consultar la tabla:

```
import pandas as pd
resultados = []

tam = [1000, 10000, 100000, 1000000, 100000000]

for t in tam:
    start = time.time()
    query = f"SELECT count() FROM benchmark_eventos WHERE usuario_id < {t}"
    res = click_client.query(query)
    end = time.time()
    resultados.append(end - start)

benchmark_df = pd.DataFrame({
    "filas_escaneadas": tam,
    "tiempo_ejecución": resultados
})

benchmark_df
```

Por último, grafiquemos los datos obtenidos.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))

plt.plot(
    benchmark_df["filas_escaneadas"],
    benchmark_df["tiempo_ejecución"],
    marker="o"
)

plt.title("Comparativa de rendimiento de las consultas en ClickHouse")
plt.xlabel("Filas filtradas")
plt.ylabel("Tiempo de ejecución (s)")
plt.grid(True)

plt.show()
```

3. Creación del Esquema de Analítica

Ahora, construiremos un sistema de analítica que se poblará a partir de los datos transaccionales de *retail_db*. Antes de crear la tabla principal de analítica, debemos cuestionarnos ¿cuál es el resultado esperado al terminar el análisis?. Para el caso del taller, identificamos que el analista de negocio requiere conocer las ventas totales por día de la semana. Este resultado será útil para determinar si se contrata o no publicidad para los fines de semana. La propuesta es presentar un resultado como el de la siguiente tabla:

Día de la semana	Facturación Total
Domingo	\$81015.46
Lunes	\$76077.14
Martes	\$85900.86
Miércoles	\$71262.4
Jueves	\$96000.93
Viernes	\$88109.91
Sábado	\$94829.3

Al terminar el taller, no solo presentaremos los datos en una tabla sino en un tablero de forma gráfica.

Tenemos en cuenta que solo nos solicitan los valores acumulados por día, es decir, la agregación se realiza por *tiempo* e incluye el valor de las ventas. La fecha la encontramos en la tabla *orders* y los valores de facturación en la tabla *order_details*.

3.1 Diseño del Esquema de Analítica

Entonces tendremos una tabla de hechos, denominada **fact_sales** que contiene la medida de las ventas procedentes de **order_details**, además de la tabla de **dim_tiempo** para el parámetro de la agregación.

La tabla de hechos fact_sales tendrá:

- **subtotal_por_producto** como la medida que contiene las ventas de ese producto en la orden de compra.
- **dim_tiempo_id** para la dimensión del tiempo.
- **dim_orden_id** para la dimensión de la orden de compra.
- **dim_product_id** para la dimensión de los productos en la orden de compra relacionada.

De forma similar, la tabla de la dimensión del tiempo tendrá los siguientes campos:

- **dim_tiempo_id** ID en segundos en formato unix timestamp
- **fecha** Fecha en formato regular Date
- **dia_semana** día de la semana.
- **dia** día del mes.
- **mes** número del mes.
- **trimestre**
- **semestre**
- **year**

Esta definición de la dimensión facilitará futuras aplicaciones y requiere poco esfuerzo su cálculo.

3.2 Creación de la Tabla de Dimensiones

 En la BD analytics, construya una tabla llamada **dim_tiempo** para almacenar la dimensión de tiempo en la base de datos *analytics* en ClickHouse con los campos definidos en la sección anterior. Utilice la documentación de tipos de datos en ClickHouse para seleccionar el tipo de dato más adecuado para los

campos [Data Types in ClickHouse](#). Aseguremos utilizar `dim_tiempo_id` como índice, es decir con la instrucción ORDER BY.

3.3 Creación de la Tabla de Hechos

 Construya el script SQL de la tabla de hechos como se diseñó anteriormente. Escriba el script en el archivo `clickhouse/init/01_schema.sql` para que se cargue una vez inicia el contenedor de ClickHouse.

3.4 Inserción en la Tabla `dim_tiempo`

Para insertar datos en la dimensión de tiempo, crearemos 365 registros que corresponden a los días del año mediante un DataFrame de pandas y luego los insertaremos en la tabla `dim_tiempo` en *analytics*:

```
# preparar la dimension de tiempo
import pandas as pd

# Rango de fechas para el año 2026
date_range = pd.date_range(start="2026-01-01", end="2026-12-31", freq="D")

# DataFrame
df_dates = pd.DataFrame({ "order_date": date_range })

# Columnas de la dimensión
dim_tiempo_df = pd.DataFrame({
    "dim_tiempo_id": df_dates["order_date"].astype("int64") // 10**9, # unix
    timestamp (seconds)
    "fecha": df_dates["order_date"].dt.date,
    "dia_semana": df_dates["order_date"].dt.dayofweek, # dia de la semana 0
    lunes, 1 martes, 6 domingo
    "dia": df_dates["order_date"].dt.day,
    "mes": df_dates["order_date"].dt.month,
    "trimestre": df_dates["order_date"].dt.quarter,
    "semestre": ((df_dates["order_date"].dt.month - 1) // 6) + 1,
    "year": df_dates["order_date"].dt.year
})
```

Ahora insertaremos el Pandas DataFrame en la base de datos analytics así:

```
import pyarrow as pa

arrow_tabla = pa.Table.from_pandas(dim_tiempo_df, preserve_index=False)

res = click_client.insert_arrow("dim_tiempo", arrow_tabla)

res.summary
```

PyArrow es la implementación para Python del proyecto Apache Arrow que proporciona un formato de datos en memoria, columnar, y eficiente para el procesamiento analítico. Arrow representa datos de columnas en

forma contigua en memoria y minimiza el I/O y es eficiente en el manejo de consultas. Arrow está diseñado independiente del lenguaje porque los datos se estructuran en memoria y pueden compartirse entre múltiples lenguajes como C++, Java, Rust y Python sin requerir serialización. Esta característica denominada *zero-copy* reduce el costo en CPU y la latencia. Acceda al detalle del proyecto Arrow en [Arrow documentación](#)

Arrow soporta otros formatos de almacenamiento columnar como Parquet o Feather para facilitar el almacenamiento comprimido y la lectura selectiva de columnas. PyArrow ofrece APIs para leer y escribir en estos formatos manteniendo la estructura columnar y la compresión. Parquet es principalmente utilizado en *data lakes* por su capacidad de lectura selectiva de columnas y la integración con varios motores analíticos. PyArrow es el intermediario entre el procesamiento en memoria y el almacenamiento persistente. Interopera con varios motores de analítica como ClickHouse, Spark, BigQuery, por esta razón es ampliamente utilizado en los pipelines modernos.

PyArrow ofrece un conjunto de operaciones vectorizadas a través del módulo `pyarrow.compute` utilizado para aplicar transformaciones, filtros y agregaciones directamente sobre los datos. Estas operaciones están optimizadas en C++ y aprovechan la ejecución vectorizada. Consulte la documentación [Documentación de funciones compute para la version 14](#)

Otra característica es el manejo explícito del esquema *schema enforcement* porque permite definir los tipos de datos y así facilita la validación y consistencia del pipeline de datos. El tipado fuerte es relevante en los sistemas analíticos porque los errores de tipo se pueden propagar silenciosamente hacia los sistemas más flexibles.

3.5 Insertar Datos en la Tabla de Hechos

Para insertar en la tabla de hechos diseñada anteriormente, ejecutaremos las siguientes tareas:

1. Extraer datos de la BD `retail_db`.
2. Crear una tabla Arrow.
3. Aplicar transformaciones (pipeline) a la tabla Arrow.
4. Insertar en la tabla **fact_sales** de **analytics**

 Escriba una consulta SQL en PostgreSQL que utiliza las tablas `order`, `order_details` y `product` retorna las columnas:

- `order_date`
- `order_id`
- `product_id`
- `unit_price`
- `discount`
- `quantity`

✅ Usando DBeaver o una celda en Jupyter notebook compruebe el funcionamiento de la consulta.

Luego ejecutamos las tareas siguientes con el script:

```
import pyarrow as pa
import pyarrow.compute as pc
from sqlalchemy import text
```

```

batch_size = 10000

with postgres_engine.connect() as conn:
    result =
conn.execution_options(stream_results=True).execute(text(postgres_query))

    columns = result.keys()

    while True:
        rows = result.fetchmany(batch_size)
        if not rows:
            break

        # -----
        # 2. CREACIÓN del batch de datos → Arrow
        # -----

        arrow_table = pa.Table.from_pylist([dict(zip(columns, row)) for row in
rows])
        display(arrow_table.slice(0, 5).to_pandas())

        # -----
        # 3. TRANSFORMACION (Arrow-native)
        # -----

        # Convertir order_date → timestamp
        order_ts = pc.cast(arrow_table["order_date"], pa.timestamp("s"))
        # dim_time_id (unix)
        dim_time_id = pc.cast(order_ts, pa.int64())

        # precio_con_descuento = unit_price - discount
        precio_con_descuento = pc.subtract(
            arrow_table["unit_price"],
            arrow_table["discount"]
        )

        # total_amount = quantity * precio_con_descuento
        total_amount = pc.multiply(
            arrow_table["quantity"],
            precio_con_descuento
        )

        # Construir la tabla final
        fact_table = pa.table({
            "dim_tiempo_id": dim_time_id,
            "dim_orden_id": arrow_table["order_id"],
            "dim_producto_id": arrow_table["product_id"],
            "subtotal_por_producto": total_amount
        })

        # prints para validación / debug
        display(fact_table.slice(0, 5).to_pandas())
        print(fact_table.schema)

```

```
# -----
# 4. CARGAR → ClickHouse (Arrow native)
# -----

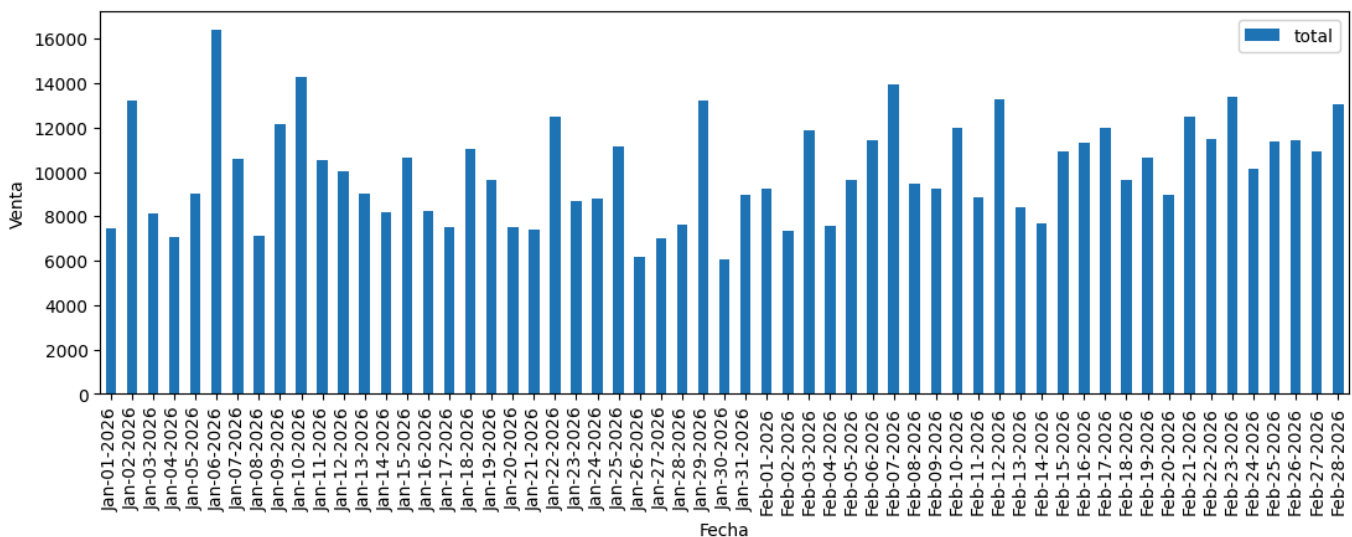
click_client.insert_arrow("fact_sales", fact_table)
```

- ✓ Compruebe que los datos están cargados en la tabla *fact_sales* de *analytics*.
- ✓ Compruebe también la estructura de directorios y archivos creados en directorio `/var/lib/clickhouse/data/` de su contenedor.

3.6 Consultar la tabla de Hechos

Después de tener datos en la tabla de hechos, podemos agregar la suma de ventas por día.

📄 Escriba una consulta SQL en ClickHouse que retorne la suma de ventas por día. Este conjunto de datos retornado es muy pequeño, y puede almacenarse en un DataFrame de Pandas y grafíquelo. El resultado deberá ser similar a:



📄 El siguiente paso consiste en consultar el conjunto de datos que responde a los requerimientos del analista de negocio. Escriba una consulta SQL en ClickHouse que utiliza las tablas *dim_tiempo* y *fact_sales* que retorne las columnas *dia_semana* y *total_ventas*. El resultado debería ser similar a:

Día de la semana	Facturación Total
Domingo	\$81015.46
Lunes	\$76077.14
Martes	\$85900.86
Miércoles	\$71262.4
Jueves	\$96000.93
Viernes	\$88109.91

Día de la semana Facturación Total

Sábado	\$94829.3
--------	-----------

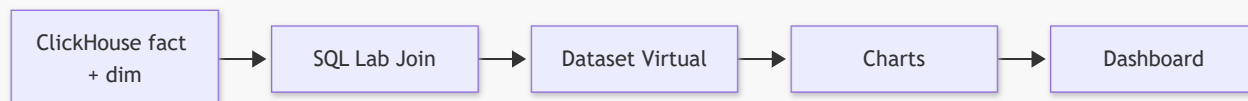
4. Construir un Dashboard en Superset

La última fase de este taller consiste en construir un dashboard en *Superset*. Apache Superset es una plataforma de visualización de datos y business intelligence (BI) que permite explorar, analizar, y visualizar datos mediante dashboards interactivos. Originalmente desarrollado por Airbnb y luego donado a Apache Software Foundation.

La arquitectura desacoplada de Superset separa la capa de almacenamiento de la de visualización y permite conectarse a múltiples motores de datos como ClickHouse, PostgreSQL, Bigquery entre otros, utilizando conectores SQLAlchemy. Su interfaz web permite crear dashboards, editar en SQL llamado *SQL Lab*, y ofrecer visualizaciones avanzadas sin programación intensiva. Superset permite crear una capa semántica ligera que permite definir datasets virtuales mediante consultas SQL y facilita el modelado analítico sin duplicar datos.

La escalabilidad y extensibilidad son las mayores características de Superset. Está diseñado para operar grandes volúmenes de datos porque delega el procesamiento a los motores subyacentes, por lo tanto, su arquitectura es distribuida. Además, soporta autenticación y control de acceso basado en roles (RBAC) y personalización mediante plugins.

Superset es una herramienta de BI que combina flexibilidad, escalabilidad y facilidad de uso. Su uso en la industria por diversas organizaciones orientadas a datos como Airbnb, Lyft o Slack lo presentan como una herramienta relevante para el ecosistema moderno de analítica.



El flujo de datos se muestra en el gráfico. Primero, una consulta a las tablas de hechos y dimensiones se construye en el SQL Lab de Superset. Luego, esta consulta se almacena como un Dataset Virtual. A partir del Dataset Virtual se construyen Charts de diferentes tipos como las clásicas barras, líneas y series de tiempo; u otros avanzados como Sankey o Chord. Por último, estos charts se insertan en un dashboard que agrupa los resultados en una sola pantalla.

4.1 Contenedor de Superset

Agregue un nuevo contenedor al compose:

```
services:
...
  superset:
    build: ./superset
    container_name: superset
    ports:
      - "8088:8088"
```

```

environment:
  SUPERSET_SECRET_KEY: supersecretkey
depends_on:
  - clickhouse_server
  - postgres_server
volumes:
  - superset_home:/app/superset_home
command: ["/app/superset_init.sh"]
...
volumes:
  superset_home:

```

El contenido de la imagen personalizada de Superset `superset/Dockerfile` es:

```

FROM apache/superset:3.1.2

USER root

COPY requirements.txt /app/requirements.txt

RUN apt-get update \
  && apt-get install -y build-essential \
  && pip install -r /app/requirements.txt

COPY superset_init.sh /app/superset_init.sh
RUN chmod +x /app/superset_init.sh

USER superset

```

El archivo de `requirements.txt` contiene las bibliotecas `clickhouse` y `sqlalchemy` para la conexión a ClickHouse. Consulte la documentación de conexión [Connect Superset to ClickHouse](#)

El contenido del archivo `superset/superset_init.sh` es:

```

#!/bin/bash

superset db upgrade

superset fab create-admin \
  --username admin \
  --firstname Admin \
  --lastname User \
  --email admin@superset.com \
  --password admin

superset init

# Add ClickHouse DB automatically
superset shell <<EOF

```

```

from superset import db
from superset.models.core import Database

db.session.add(Database(
    database_name="ClickHouse",

sqlalchemy_uri="clickhouse+http://admin:admin123@clickhouse_server:8123/analytics"
))
db.session.commit()
EOF

superset run -h 0.0.0.0 -p 8088

```

Lance el compose con todos los servicios y contenedores, e inicie la construcción del dashboard.

4.2 Conexión de Superset a Clickhouse

1. Compruebe la conexión desde Superset a Clickhouse: desde la terminal del contenedor pruebe `curl -u admin:admin123 "http://clickhouse-server:8123/?query=SELECT%201"`
2. Abra un navegador y conéctese al servicio de Superset en el puerto expuesto para el servicio.
3. Establezca una conexión a base de datos.
 1. Settings > Edit Database > Otro
 2. SQLAlchemy URI con el formato `clickhouse+http://{user}:{password}@{host}:{port}/{database}`
 3. Compruebe que la conexión esté funcionando correctamente
 4. Guarde la conexión a la base de datos.

4.3 Construir Datasets

1. Click en la pestaña Datasets de la barra del menú principal.
2. Construya un dataset para la tabla `fact_sales`.
 1. Establezca la métrica correspondiente a la agregación realiza en secciones anteriores.
3. Construya otro dataset para la tabla `dim_tiempo`.

4.4 Construir Virtual Datasets

Los virtual datasets son definidos mediante una consulta SQL en vez de una tabla física de la tarea anterior. Un virtual dataset no contiene datos propios, sino que es una capa lógica de una consulta y la expone como una tabla reutilizable. Este enfoque es similar a las *vistas* de una base de datos relacional, pero orientado a la capa de analítica y visualización.

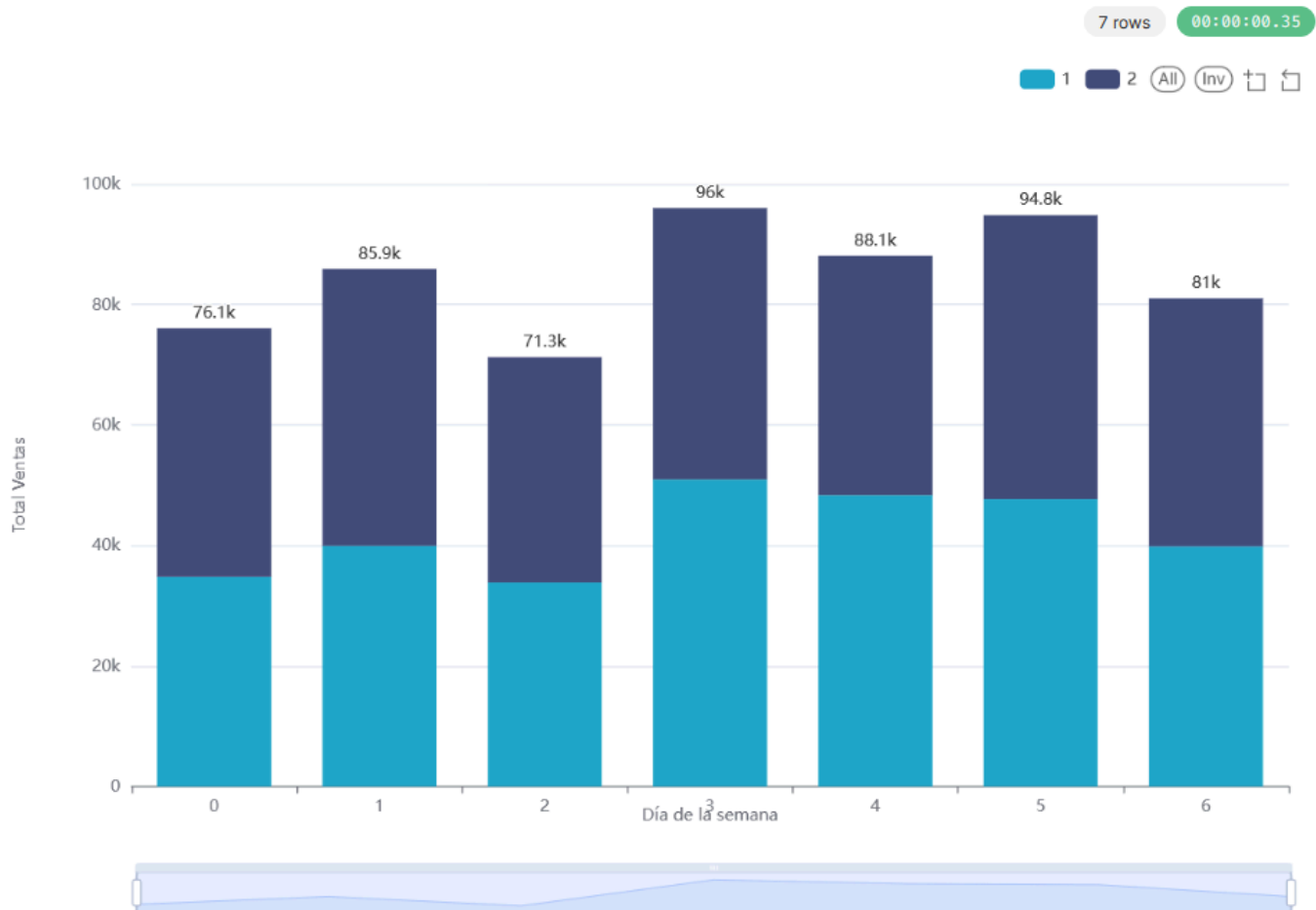
1. Construya una consulta JOIN de `fact_sales` y `dim_tiempo`
 - fecha,
 - año,
 - mes,
 - día,
 - día de la semana,
 - subtotal_por_producto
 - semestre, trimestre, mes, día, dim_orden_id, dim_producto_id, subtotal_por_producto

2. Guarde el dataset

4.5 Crear gráficos

1. Charts > nuevo chart
2. Seleccione Bar Chart
 1. x-axis = dia_semana
 2. Métrica = SUM(subtotal_por_producto)
 3. Dimension = mes

El resultado del gráfico sería similar al siguiente:

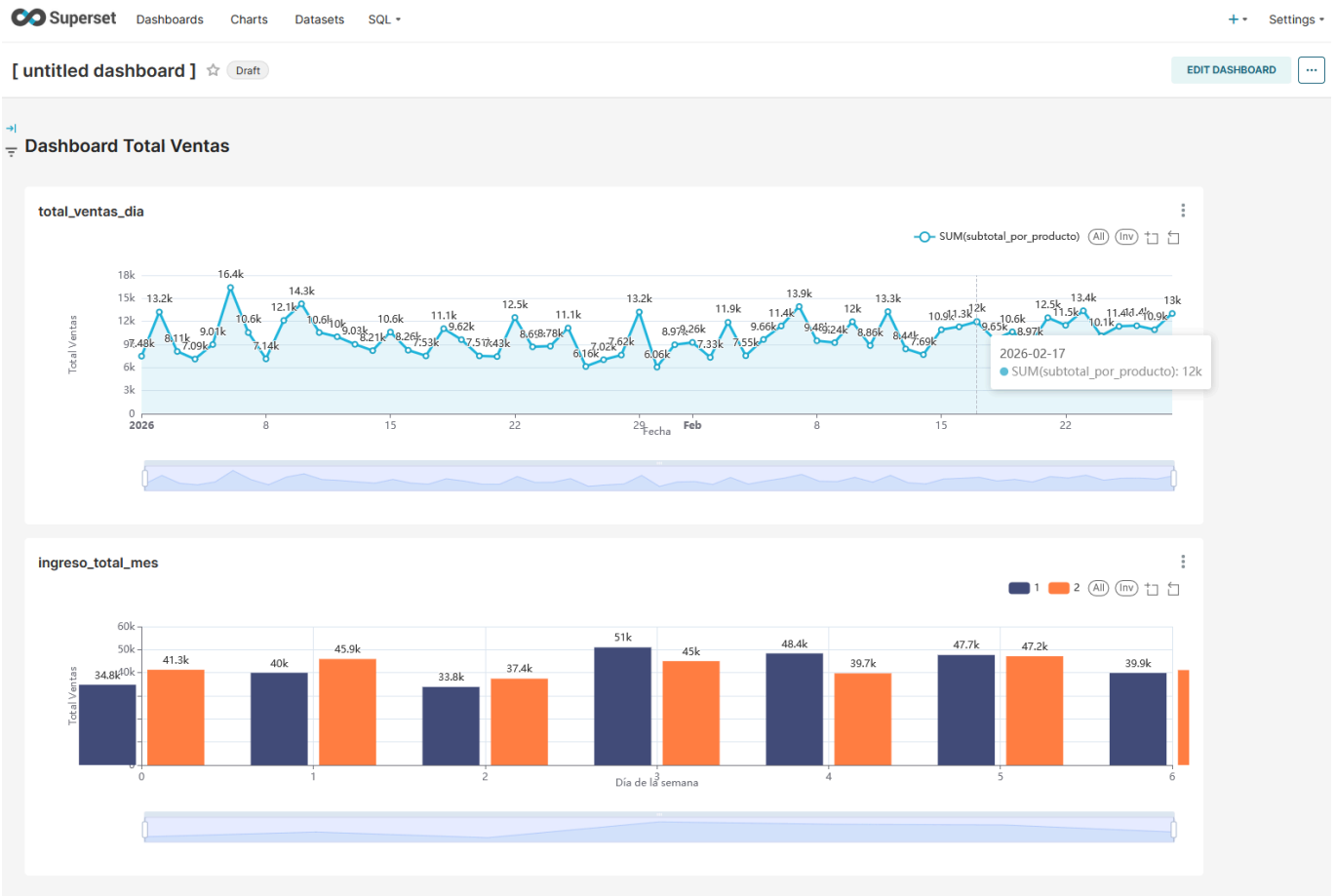


4.6 Crear dashboard

Por último, construya un dashboard que contiene la información solicitada por el analista de negocio más un gráfico de las ventas totales por día.

1. Dashboards > nuevo dashboard
2. Agregue los Charts al dashboard

El resultado del gráfico sería similar al siguiente



Ejercicios Opcionales

1. El analista de negocio quiere conocer cuanto es el valor de ventas en los días festivos, adicionales al domingo. ¿cómo resolvería el problema?
2. El analista de negocio requiere identificar las categorías de productos que representan el 50% del total de ventas. El resultado es una gráfica que muestra únicamente las categorías de productos ordenados por ventas superior al 50%.
3. El analista de negocio quiere agrupar clientes similares según sus ítems comprados. Para un periodo de observación medido en semanas 1 – 52. Usando la distancia de Jaccard y la similitud del coseno.
4. El analista de negocio requiere consultar la *rentabilidad* de producto, por región y territorio. Para un periodo en estudio *ini_date*, *end_date*.

Referencias

- Abadi, D. (2008). Query Execution in Column-Oriented Database Systems. MIT
- Abadi, D. et al. (2009). Column-Oriented Database Systems. Foundations and Trends in Databases.
- Boncz, P. et al. (2005). MonetDB/X100: Hyper-Pipelining Query Execution. CIDR
- Kimball & Ross (2013). The Data Warehouse Toolkit
- Lemire, D. et al. (2015). Decoding billions of integers per second through vectorization
- Neumann, T. (2011). Efficiently Compiling Efficient Query Plans for Modern Hardware. VLDB
- Stonebraker et al. (2007). The End of an Architectural Era